Perangkat Lunak Matematika

Nama : Addien Zakia Hidayat
NRP : 5002231063

Tugas 1

Aturan Pengerjaan:

- Tuliskan identitas Anda dengan urutan: Nama, NRP.
- Telusuri program yang Anda buat dan jelaskan output yang dihasilkan pada program yang dibuat.
- Kirimkan file jawaban Anda dalam ekstensi .pdf dengan format nama "Tugas 1-PLM-Nama" paling lambat sebelum kelas dimulai. Keterlambatan pengumpulan jawaban akan berakibat pada penalti nilai. Pengumpulan file jawaban yang terlambat lebih dari satu hari akan dianggap tidak mengerjakan.

In [3]:
```
#Tuliskan library yang Anda perlukan disini
import math
```

Nomor 1

Jelaskan jenis-jenis error di Python, dan berikan contohnya disertai dengan cara menangani error tersebut.

(Jawaban ada di bawah)

Nomor 2

Buatlah program yang dapat menampilkan output berupa luas dari segitiga sama sisi dengan panjang sisi sebesar 12.

In [4]:
```
## Tuliskan jawaban Anda disini
alas = 12
tinggi = 12
```

```
luas =  0.5 * alas * tinggi

print("Luas alas segitiga sama sisi dengan alas", alas, "dan tinggi", tinggi, "adalah:", luas)
```

```
Luas alas segitiga sama sisi dengan alas 12 dan tinggi 12 adalah: 72.0
```

Nomor 3

Buatlah program yang dapat menampilkan output berupa luas dan keliling lingkaran dengan jari-jari 8.

In [5]:
```
## Tuliskan jawaban Anda disini
jari_jari = 8
luas = 22 / 7 * jari_jari * jari_jari
keliling = 22 / 7 * (2 * jari_jari)

print("Luas lingkaran dengan jari-jari", jari_jari, "adalah", luas)
print("Keliling lingkaran dengan jari-jari", jari_jari, "adalah", keliling)
```

```
Luas lingkaran dengan jari-jari 8 adalah 201.14285714285714
Keliling lingkaran dengan jari-jari 8 adalah 50.285714285714285
```

Nomor 4

Buatlah program yang dapat menampilkan output berupa akar-akar dari $z^6 = 8$. (Petunjuk: Ingat kembali materi mencari akar dari bilangan kompleks.)

$$z^6 = 8$$
$$z_k = \sqrt{2}\,\mathrm{cis}\left(\frac{k\pi}{3}\right)$$
$$n = 6; k = 0, 1, 2, 3, 4, 5$$

In [6]:
```
## Tuliskan jawaban Anda disini
for i in range (6):
    z = math.sqrt(2) * (complex(math.cos((i * math.pi) / 3), math.sin((i * math.pi) / 3)))
    print(f"Akar dari z{i} adalah", z)
```

```
Akar dari z0 adalah (1.4142135623730951+0j)
Akar dari z1 adalah (0.7071067811865477+1.2247448713915892j)
Akar dari z2 adalah (-0.7071067811865472+1.2247448713915892j)
Akar dari z3 adalah (-1.4142135623730951+1.7319121124709868e-16j)
Akar dari z4 adalah (-0.7071067811865482-1.224744871391589j)
Akar dari z5 adalah (0.7071067811865477-1.2247448713915892j)
```

Nomor 5

Buatlah program yang dapat menampilkan output berupa 10 bilangan pertama dari barisan Fibonacci

$$f_n = f_{n-1} + f_{n-2}.$$

(Petunjuk: $f_0 = f_1 = 1, f_2 = 2$ dengan f_n adalah bilangan Fibonacci ke-n.)

In [7]:
```python
## Tuliskan jawaban Anda disini
def fibonacci(n):
    if n == 0 or n == 1:
        return 1
    elif n == 2:
        return 2
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = fibonacci(10)
print(n)
```

```
89
```

Nomor 6

Buatlah program yang dapat menampilkan output berupa 5 jumlahan parsial pertama dari deret

$$\sum_{n=0}^{\infty} \frac{1}{n^2}.$$

In [12]:
```python
## Tuliskan jawaban Anda disini
jumlah = 0
```

```python
for i in range(5):
    if i == 0:
        jumlah += 0
    else:
        jumlah += 1 / (i**2)

print(jumlah)
```

```
1.4236111111111112
```

Nomor 7

Buatlah program yang dapat menampilkan output berupa bentuk bilangan desimal dari bilangan biner

$$1111111111111000000000000.$$

Apa fakta menarik yang Anda ketahui tentang bilangan ini?

In [9]:
```python
## Tuliskan jawaban Anda disini
int("1111111111111000000000000", 2) # based on 2 elements (1 and 0)

# (2^13 - 1) * 2^12
```

Out[9]:
```
33550336
```

Fakta menarik terkait hal ini yaitu

Pada umumnya ketika kita melakukan konversi angka biner ke suatu angka biasa adalah dengan cara `mengalikan angka dari kanan ke kiri dengan 2^k untuk k = 0, 1, 2, ..., n dan dilakukan penjumlahan untuk masing-masing hasil perkalian`

contoh:

```
angka biner: 101

-> (1 x 2^2) + (0 x 2^1) + (1 x 2^0)
= 4 + 0 + 1
= 5
```

akan tetapi terdapat pola dalam angka biner yang dapat mempermudah dalam melakukan perhitungan ke dalam sebuah angka readable

ketika terdapat angka biner dengan n angka 1 berderet di depan bilangan, kita dapat dengan mudah menghitung dengan rumusan `2^n - 1`

contoh:

```
biner: 1111

n = 4
-> (2^n - 1)
= (2^4 - 1)
= 16 - 1 = 15
```

selain itu ketika terdapat n angka 0 di belakang angka biner, kita dapat dengan mudah menghitung dengan mengkalikan angka depan biner dengan (2^n) dengan syarat `angka depan merupakan n angka 1 pada biner`

contoh:

```
1110000

111  -> n = 3, (2^3 - 1) = 8 - 1 = 7
0000 -> n = 4, (2^4) = 16

1110000 = 7 x 16 = 112
```

akan tetapi ketika terdapat gabungan angka seperti `11101000`, maka angka biner ini tidak termasuk pola yang disebutkan diatas sehingga untuk melakukan konversi ke angka biasa dilakukan dengan cara pada umumnya.

Nomor 8

Buatlah program yang dapat menampilkan output berupa titik berat dari bidang datar segitiga dengan titik sudut di $(0,0), (4,6), (4,-6)$. (Petunjuk: Ingat kembali materi titik berat di Kalkulus 2)

In [10]:
```python
## Tuliskan jawaban Anda disini

A, B, C = [[0, 0], [4, 6], [4, -6]]

Dx = ((A[0] + B[0] + C[0]) / 3)
Dy = ((A[1] + B[1] + C[1]) / 3)

print(f"Sehingga titik berat dari bidang datar segitiga adalah ({Dx:.2f}, {Dy:.2f})")
```

```
Sehingga titik berat dari bidang datar segitiga adalah (2.67, 0.00)
```

Nomor 1

Jenis-jenis Error dalam Python

- **Syntax Error**

Terjadi karena kesalahan penulisan kode

Contoh:

```python
# --- Contoh Salah ---
if 5 > 3
    print("Benar")

# --- Contoh Benar ---
if 5 > 3:
    print("Benar")
```

Program bakalan error jika terdapat *Syntax Error* seperti berikut:

```
  File "d:\PLM\r.py", line 1
    if 5 > 3
             ^
SyntaxError: expected ':'
```

- **Indentation Error**

Python sangat sensitif terhadap indentasi pada program yang dibuat sehingga jika terdapat program yang memiliki kesalahan terdapat indentasi maka akan muncul error *Indentation Error*

Contoh:

```
# --- Contoh Salah ---
if 5 > 3:
print("Benar")

# --- Contoh Benar ---
if 5 > 3:
    print("Benar")
```

Program bakalan error jika terdapat *Indentation Error* seperti berikut:

```
  File "d:\PLM\r.py", line 2
    print("Benar")
    ^
IndentationError: expected an indented block
after 'if' statement on line 1
```

- **Name Error**

Biasanya terjadi ketika terdapat suatu variable yang dipanggil sebelum variable tersebut dideklarasikan terlebih dahulu

Contoh:

```
# --- Contoh Salah ---
print(x)

# --- Contoh Benar ---
x = "Benar"
print(x)
```

Program bakalan error jika terdapat *Name Error* seperti berikut:

```
Traceback (most recent call last):
  File "d:\PLM\r.py", line 1, in <module>
    print(x)
```

```
            ^
NameError: name 'x' is not defined
```

- **Type Error**

Terjadi ketika tipe data yang tidak sesuai

Contoh:

```python
# --- Contoh Salah ---
print("Umur: " + 20)

# --- Contoh Benar ---
# dilakukan konversi tipe data dari int ke string
print("Umur: " + str(20))
```

Program bakalan error jika terdapat *Type Error* seperti berikut:

```
Traceback (most recent call last):
  File "d:\PLM\r.py", line 1, in <module>
    print("Umur: " + 20)
          ~~~~~~~~~^~~~
TypeError: can only concatenate str (not "int") to str
```

- **Value Error**

Terjadi ketika tipe data benar tetapi nilainya tidak sesuai

Contoh:

```python
# --- Contoh Salah ---
x = int("abc")

# --- Contoh Benar ---
x = int("123") # output: 123 dengan type int
```

Program bakalan error jika terdapat *Value Error* seperti berikut:

```
Traceback (most recent call last):
  File "d:\PLM\r.py", line 1, in <module>
```

```
    x = int('abc')
        ^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'
```

- **Index Error**

Terjadi ketika mengakses indeks list yang tidak ada

Contoh:

```python
# --- Contoh Salah ---
data = [1, 2, 3]
print(data[5])

# --- Contoh Benar ---
data = [1, 2, 3]
print(data[1]) # output: 2
```

Program bakalan error jika terdapat *Index Error* seperti berikut:

```
Traceback (most recent call last):
  File "d:\PLM\r.py", line 2, in <module>
    print(data[5])
          ~~~~^^^
IndexError: list index out of range
```

- **Key Error**

Terjadi ketika mengakses key dictionary yang tidak ada

Contoh:

```python
# --- Contoh Salah ---
data = {"nama": "Budi"}

# key 'umur' tidak  ada pada dict data
print(data["umur"])


# --- Contoh Benar ---
```

```python
data = {"nama": "Budi"}

print(data["nama"]) # output: Budi
# atau
print(data.get("nama")) # output: Budi
```

Program bakalan error jika terdapat *Key Error* seperti berikut:

```
Traceback (most recent call last):
  File "d:\PLM\r.py", line 2, in <module>
    print(data["umur"])
          ~~~~^^^^^^^^^
KeyError: 'umur'
```

- **Zero Division Error**

Terjadi ketika membagi dengan nol

Contoh:

```python
# --- Contoh Salah ---
print(10 / 0)
```

Program bakalan error jika terdapat *Zero Division Error* seperti berikut:

```
Traceback (most recent call last):
  File "d:\PLM\r.py", line 1, in <module>
    print(10 / 0)
          ~~~^~~
ZeroDivisionError: division by zero
```

Error Handling

Terkadang ketika membuat suatu program terdapat beberapa error yang muncul pada program dan biasanya dari error tersebut kita tidak mau muncul begitu saja pada output terminal, sehingga kita bisa melakukan `Error Handling` pada Python menggunakan `try-catch` untuk mengatasi hal tersebut

Struktur dasar `try-catch`:

```python
try:
    # Error code
except JenisError: # -> Jenis error yang mau dihandle
    # Error handler
else:
    # dijalankan jika tidak ada error
finally:
    # Log ketika program berhasil dijalankan
```

Contoh:

```python
try:
    angka = int(input("Masukkan angka: "))
    hasil = 10 / angka
except ValueError:
    print("Input harus angka")
except ZeroDivisionError:
    print("Tidak boleh nol")
else:
    print("Hasil:", hasil)
finally:
    print("Program selesai")
```